

Database Systems

1. The Relational Model

A relational database organizes data into tables. Each table, also called a relation, is made up of rows and columns. A row represents a single record, and a column represents an attribute of that record.

The relational model was proposed by E. F. Codd in 1970 as a way to separate the logical structure of data from the way it happens to be stored on disk. Users work with tables and the operations of relational algebra rather than with physical storage details.

A primary key is a column or set of columns whose values uniquely identify each row in a table. A foreign key is a column that refers to the primary key of another table and is used to create a relationship between the two tables.

Because a primary key must be unique and non-null, it guarantees that every row in a table can be identified unambiguously. The foreign key is the mechanism that implements relationships: a row in one table can point to a row in another table through the value of the foreign key column.

2. SQL and Queries

Structured Query Language, or SQL, is the standard language for interacting with relational databases. A SELECT statement retrieves data from one or more tables and can filter rows with a WHERE clause.

A basic query names the columns to return, the table to read from, and any conditions the rows must satisfy. The database system optimises the query, plans an execution strategy, and returns the matching rows. The same declarative query returns the same logical result regardless of how the database physically executes it.

Joins combine rows from two tables based on a related column, such as a foreign key. An INNER JOIN returns only rows that match in both tables, while a LEFT JOIN also returns all rows from the left table, even those with no match on the right.

Joining is one of the most powerful and most used operations in relational databases because it lets information stored in different tables be combined into a single result. Deciding which join type to use depends on which rows should be kept when there is no matching row on the other side.

3. Normalization and Design

Normalization is the process of organizing the columns and tables of a database to reduce redundant data and improve data integrity. The goal is to eliminate duplicate information so that each fact is stored in exactly one place.

Redundant data is a problem because it wastes space and, more importantly, because it creates the risk of inconsistency: the same fact copied in several places can be updated in one place but not the others. Normalization reduces this risk by structuring the tables so each fact appears once.

A design is said to be in first normal form when every column holds atomic, indivisible values and there are no repeating groups. Removing partial and transitive dependencies moves a design toward higher normal forms.

Second normal form removes partial dependencies, where a column depends on only part of a composite key. Third normal form removes transitive dependencies, where a column depends on another non-key column rather than directly on the key. Each step reduces the ways in which redundant data can be introduced.

4. Transactions and ACID

The four ACID properties are atomicity, consistency, isolation, and durability. Together they guarantee that a database remains correct in the face of failures and concurrency. Atomicity keeps each transaction all-or-nothing, consistency keeps the data valid, isolation keeps concurrent transactions separate, and durability keeps committed changes permanent.

A transaction is a sequence of database operations that must be executed as a single unit. Either all of the operations complete, or none of them take effect. Transactions are committed to make their changes permanent, or rolled back to undo them.

Transactions protect the database from failures and from concurrent conflicts. If a failure occurs partway through a transaction, the database restores the state to what it was before the transaction began, so a partially applied change never becomes visible.

The ACID properties describe what makes a transaction reliable. Atomicity means the transaction is all-or-nothing. Consistency means the database moves from one valid state to another. Isolation means concurrent transactions do not interfere with each other. Durability means committed changes survive a system failure.

Atomicity is achieved by logging the operations of the transaction so they can be undone if the transaction never completes. Durability is achieved by writing committed changes to stable storage before acknowledging success, so the change is not lost even if the machine loses power afterward. Consistency is enforced by the database and the application together: the database checks constraints such as uniqueness and foreign-key integrity so that no commit can leave the data in an invalid state.

Isolation is the property that stops two transactions running at the same time from observing each other's partial work. Without isolation, one transaction could read a value that another transaction later rolls back, producing a result that could not occur if the transactions had run one after the other. Strong isolation guarantees serializable behaviour, while weaker isolation levels permit more concurrency in exchange for accepting some anomalies.